

LC 797: All paths from source to Target

Authored by: Arpit Raj, LNMIIT Jaipur | Date: July 30, 2026

Amazon

Adobe

Problem Overview

- (nodes labeled from 0 to $n-1$)
- need to ret all possible paths from 0 to $n-1$
- **[DAG]** (Directed Acyclic Graph)

0 → 1

↓ ↓

2 → 3

then → $[[0,1,3], [0,2,3]]$

lets say we went from 0 to 1 then in order to see the path of 0 to 2 we must come back to 0, hence

Approach: [DFS + Backtrack]

cant use BFS → BFS can find path, but i need every path, DFS explores one complete path before coming back via backtrack

- **No need of seen array** because graph is Directed acyclic, i.e no cycles

Algorithm & Complexity

- **Algo** → maintain a path array and ans array for ans.push(path)
- let P valid paths, with length at most V then copying each path = $O(V)$ so $O(PV)$

C++ Implementation

```
class Solution {
public:
    vector<int> path;
    vector<vector<int>> ans;

    void dfs(vector<vector<int>>& graph, int curr, int dest) {
        //SEB principle
        //No need of S=seen , as DAG
        path.push_back(curr);
        if(curr == dest) {
            ans.push_back(path);
            path.pop_back();
            return; //try to explore more paths now
        }

        for(auto &neigh : graph[curr]) {
            dfs(graph, neigh, dest); //explore
        }

        path.pop_back(); //backtrack;
    }

    vector<vector<int>> allPathsSourceTarget(vector<vector<int>>& graph) {
        int n = graph.size();
        dfs(graph, 0, n - 1);
        return ans;
    }
};
```

